

FAZ 1

Environment Setup Your First Website

Geliştirme araçlarından tarayıcı motoruna ve yayına

Uyarı: Bu içerik, SCÜ Şarkışla UBYO Web Programlama I dersi kapsamında tamamen eğitim amaçlı çevrilmiş ve derlenmiştir. Orijinal dokümantasyon kaynakları (MDN Web Docs vb.) kendi orijinal lisanslarına (CC-BY-SA, MIT) tabidir. Bu çalışmanın hiçbir ticari amacı yoktur.

Sunumun yol haritası

Geliştirme ortamı

Editör, dosyalar, yerel sunucu, terminal ve Git. Bu sıra, bir ziyaretçinin gördüğü sayfayı hazırlarken geliştiricinin hangi adımları geçtiğini gösterir.

Web'in çalışma yolu

DNS, TCP, HTTP yanıtı ve tarayıcının çizim süreci. Bir sayfanın açılması yalnızca dosyayı çalıştırmak değildir; ağ ve tarayıcı birlikte çalışır.

İlk site ve yayın

HTML, CSS, JavaScript ve sitenin internete açılması. Son slaytlarda bütün parçaları tek bir örnek üzerinden birleştireceğiz.

01 / Geliştirme ortamı

Kod yazmadan önce araçların ne yaptığını anlayalım.

Neden Word yerine kod editörü?

Word belgeyi biçimlendirir

Kalınlık, sayfa düzeni ve yazı tipi gibi bilgiler belgeye eklenir. Bu biçim, kaynak kodun beklediği düz metin yapısına uymaz. Örneğin Word'deki görünmez biçim bilgisi, HTML etiketinin doğrudan okunması gereken dosya yapısıyla karışır.

Kod dosyası düz metindir

Bir .html dosyası etiketi, boşlukları ve karakterleri açıkça içerir. Editör bu metni kaydeder; tarayıcı da bu dosyayı okur. Bir etiket yazdığında `<h1>` karakterleri aynen dosyada kalmalıdır; ekranda görünen bir başlık biçimi olarak saklanmamalıdır.

Editörün katkısı

Sözdizimi renklendirme, otomatik tamamlama, arama ve hata işaretleri geliştiricinin kodu takip etmesini kolaylaştırır. Bu araçlar kodu senin yerine çalıştırmaz; hata ihtimalini yazarken görmeni ve dosyaları hızlı bulmanı sağlar.

Örnek: index.html dosyasını gerçek kod dosyası olarak kaydet.

Editör kaputun altında ne yapar?

Metin ve sözdizimi

Editör dosyayı karakterler olarak açar. Dil desteği bu karakterleri etiket, anahtar sözcük ve ifade gibi parçalara ayırır. Örneğin açılmamış bir parantezin işaretlenmesi, kodun bu parçalara ayrıştırılması sayesinde mümkündür.

AST ne zaman oluşur?

Bazı dil sunucuları ve eklentiler kodu ayrıştırıp Abstract Syntax Tree (AST) kurar. Her editör özelliği AST gerektirmez. Bu ağaç, iç içe ifadelerin birbirleriyle ilişkisini gösterir; 'bu değişken nerede tanımlandı?' gibi sorulara yanıt verir.

Eklentiler

VS Code gibi editörlerde Extension Host eklentileri ayrı bir süreçte çalıştırabilir. Bu ayırım editörün yanıt verebilirliğine yardımcı olur; sınırsız RAM koruması sağlamaz. Çok ağır bir eklenti yine bellek tüketebilir; ayrı süreç kullanımı onu otomatik olarak güvenli veya hafif yapmaz.

Dosyalar ve klasörler gerçekte nedir?

Klasör bir eşlemedir

Dosya sistemi klasörde adları ilgili dosya kayıtlarına bağlar. İçerik, yalnızca klasör adının içinde saklanan bir metin değildir. Bu nedenle klasörü taşımak, dosyanın adını değiştirmek ve içeriğini değiştirmek farklı dosya sistemi işlemleridir.

Inode örneği

Linux ext4'te inode dosyanın boyutu, izinleri ve veri bloklarına ilişkin bilgileri tutar. Klasör girdisi dosya adını inode numarasıyla ilişkilendirir. Inode bir RAM adresi değildir; diskteki dosya sisteminin kullandığı bir kayıttır ve gerektiğinde belleğe alınır.

Proje düzeni

index.html, styles/style.css ve images/logo.png gibi anlaşılır bir yapı bağlantı hatalarını azaltır. Örneğin HTML içindeki styles/style.css yolu klasör yapısında gerçekten o dosyaya karşılık gelmelidir.

Dosya adları neden önemlidir?

Boşluk ve URL

Boşluk URL içinde genellikle %20 olarak kodlanır. Bu tek başına hata değildir; yanlış yazılan yol dosyanın bulunamamasına neden olur. Örneğin 'ekip fotoğrafı.png' adres çubuğunda 'ekip %20fotoğrafı.png' biçiminde görünebilir; kodlama normaldir.

Büyük ve küçük harf

Windows'taki yaygın NTFS kullanımında dosya adları çoğunlukla harfe duyarsızdır. Linux ext4 ise genellikle Logo.png ile logo.png adlarını farklı kabul eder. Kodda images/Logo.png yazıp sunucuda images/logo.png tutarsan yerelde çalışan görsel canlı sitede kaybolabilir.

Pratik kural

Küçük harf ve kısa çizgi kullan: ekip-fotografi.png. Dosya bağlantılarında adı birebir eşleştirebilir. Türkçe karakterlerden kaçınmak da URL paylaşımı ve farklı araçlarla çalışma sırasında işleri sadeleştirir.

file:// ile localhost arasındaki fark

file://

Tarayıcı dosyayı doğrudan diskten açar. Köken ve güvenlik kuralları yüzünden fetch, modül veya başka dosyalara erişim beklenmedik biçimde kısıtlanabilir. Örneğin HTML'yi çift tıklayarak açınca adres file:/// ile başlar ve fetch('data.json') isteği engellenebilir.

Yerel sunucu

http://localhost adresi bilgisayardaki HTTP sunucusuna gider. 127.0.0.1 loopback adresi trafiği aynı makineye geri yönlendirir. Yerel sunucu bir programdır; 127.0.0.1'e gelen isteği karşılar ve dosyayı HTTP yanıtı olarak gönderir.

Neden kullanılır?

Siteyi HTTP üzerinden geliştirmek, yayındaki çalışma biçimine yaklaşır ve tarayıcı güvenlik kurallarını daha tutarlı test etmeyi sağlar. Tarayıcının same-origin ve CORS kurallarını geliştirme sırasında daha gerçekçi biçimde gözlemleyebilirsiniz.

Terminal işletim sistemiyle nasıl konuşur?

Shell

Komutu okur, değişkenleri ve yolları çözümler, ardından ilgili programı başlatır. Programlar işletim sistemi hizmetleri için sistem çağrıları kullanır. Örneğin 'git status' yazdığında shell git programını bulup başlatır; git de dosya bilgileri için işletim sisteminden hizmet alır.

stdin ve stdout

Bir program girdiyi standart girişten alır, sonucu standart çıkışa yazar. Boru işareti (|) bir programın çıktısını diğerinin girdisine bağlar. 'cat dosya.txt | sort' komutunda ilk programın ürettiği satırlar ikinci programa giriş olur.

PATH

Komut adını yazdığında shell PATH içindeki klasörlerde çalıştırılabilir programı arar. Bulamazsa 'command not found' benzeri hata alırsın. Git kurulu olsa bile PATH'te yer almıyorsa terminal komutu bulamaz; tam yolu veya PATH ayarını düzeltirsin.

Git: Sürüm geçmişi nasıl kurulur?

.git dizini

Depodaki nesneler, referanslar ve geçmiş bilgileri burada tutulur. Commit, belirli bir andaki proje durumuna ve önceki commitlere bağlanır. Bu yüzden .git klasörünü silmek çalışma dosyalarını bıraksa bile yerel geçmişi ve dal bilgilerini kaldırır.

Hash ve DAG

Git nesneleri içerik karmalarıyla tanımlar; SHA-1 şifreleme değildir. Commit ilişkileri döngüsüz yönlü bir grafik (DAG) oluşturur. İçerik değişince karma değeri de değişir; hash, veriyi geri açabilen bir şifre değildir.

İş akışı

Değişikliği ekle, commit oluştur, gerektiğinde önceki sürümü incele. Yeni depo biçimlerinde SHA-256 desteği de bulunur. Commit oluşturmak geçmişte bir durak kaydeder. Dal ise bu duraklardan birini gösteren hareketli bir işarettir.

02 / Ağ iletişimi

Bir adres yazıldığında dosyalar tarayıcıya nasıl ulaşır?

Adres yazınca ilk adım: DNS

Önbellek kontrolü

Tarayıcı ve işletim sistemi yakın zamanda öğrenilen adresleri kullanabilir. Modem veya yerel çözümleyici de önbelleğe sahip olabilir. Örneğin demirdagsiber.com yazdığında tarayıcı önce bu ismin hangi IP adresine karşılık geldiğini öğrenmelidir.

Çözümleyici

Yanıt yoksa yapılandırılmış DNS çözümleyicisi alan adını sorgular; gerektiğinde kök, üst düzey alan ve yetkili sunucular üzerinden yanıt bulunur. Kök sunucu doğrudan her sitenin IP'sini bilmez; sorguyu ilgili üst düzey alan ve yetkili kaynağa yönlendirir.

Sonuç

Alan adının IP adresi elde edilir. Önceki yanıtın TTL süresi bitene kadar önbellekte kalması mümkündür. Alan adı değiştiğinde bazı ziyaretçilerin eski IP'yi görmesinin nedeni çoğu kez önbellekteki TTL'dir.

Bağlantı: TCP ve HTTPS

TCP üçlü el sıkışma

TCP kullanıldığında istemci SYN, sunucu SYN-ACK, istemci ACK göndererek bağlantıyı kurar. SYN 'bağlanmak istiyorum', SYN-ACK 'hazırım', ACK ise 'yanıtını aldım' anlamına gelen üç adımdır.

TLS aşaması

HTTPS için istemci ve sunucu ayrıca TLS üzerinden kimliği ve şifreli iletişim anahtarlarını kararlaştırır. Bu aşamadan sonra tarayıcı sertifikayı doğrular; doğru sunucuyla konuştuğundan emin olmayı hedefler.

HTTP isteği

Tarayıcı / gibi bir yolu ister; sunucu durum kodu, başlıklar ve içerikle yanıt verir. HTTP/3 gibi bağlantılar TCP yerine QUIC kullanabilir. Dolayısıyla tüm modern web bağlantıları TCP kullanır demek doğru değildir; HTTP/3 önemli bir istisnadır.

Veri paketleri nasıl gelir?

Parçalara ayrılma

Büyük veri ağ üzerinde küçük paketler halinde taşınır. Paketler ağ koşullarına göre farklı yönlendiricilerden geçebilir. Örneğin büyük bir görsel tek bir parça halinde taşınmak yerine ağın taşıyabileceği küçük birimlere bölünür.

Doğru sıraya koyma

TCP eksik veriyi yeniden isteme ve sıraya koyma işlerini üstlenir. Tarayıcı tamamlanan yanıtı HTML, CSS veya görsel olarak işler. Paketlerin farklı yollardan ulaşması uygulamanın içeriği karışık göstereceği anlamına gelmez; taşıma katmanı sırayı yönetir.

Gecikmenin kaynağı

DNS, bağlantı kurma, ağ mesafesi ve sunucunun yanıt süresi sayfanın açılma hızını etkiler. Uzak sunucudaki aynı sayfanın yakın sunucuya göre geç açılmasının nedenlerinden biri ağ gecikmesidir.

Arama motoru botları sayfayı nasıl bulur?

Keşif ve tarama

Botlar bağlantıları ve site haritalarını izleyerek URL'leri bulur; izin verilen sayfaları isteyip içeriği okur. Örneğin ana sayfadaki bir bağlantı botun daha önce bilmediği proje sayfasını keşfetmesini sağlayabilir.

İçeriği yorumlama

Başlıklar, bağlantılar ve semantik HTML sayfanın konusunu anlamaya yardım eder. Bazı botlar JavaScript çalıştırarak sonradan gelen içeriği de görebilir. `<h1>` başlığı ve `<main>` alanı insanlara olduğu gibi arama sistemlerine de içerik düzeni hakkında ipucu verir.

Dizinleme

Arama motoru uygun içeriği kendi dizinine ekleyebilir; taranmak otomatik olarak dizine alınmak anlamına gelmez. Giriş gerektiren, engellenmiş veya düşük değerli sayfalar bulunmuş olsa bile arama sonuçlarına eklenmeyebilir.

03 / Tarayıcı motoru

HTML ve CSS ekranda gördüğümüz sayfaya nasıl dönüşür?

HTML'den DOM ağacına

HTML ayrıştırıcısı

Tarayıcı gelen baytları karakterlere ve HTML tokenlarına dönüştürür; açılış ve kapanış etiketlerini işleyerek düğümler kurar. Örneğin `<main><h1>Merhaba</h1></main>` yapısı bellekte iç içe düğümlere dönüşür.

DOM

Document Object Model, sayfanın bellekteki ağaç temsilidir. JavaScript bu ağaçtaki öğeleri bulup değiştirebilir. Bu ağaç, sayfanın yalnızca ekrandaki görünümü değildir; öğeler arasında ebeveyn ve çocuk ilişkisi kurar.

Derleme ifadesi

HTML bir programlama dili gibi makine koduna derlenmez. Tarayıcı onu bir ayrıştırıcıyla işler; 'yorumlanır' demek burada kabaca açıklayıcı bir ifadedir. HTML hataları olsa bile tarayıcı çoğu zaman toparlamaya çalışır; bu yüzden tarayıcıda görünmesi kodun doğru olduğu anlamına gelmez.

DOCTYPE ve Quirks Mode

Standart mod

En baştaki `<!DOCTYPE html>` modern HTML belgesini işaretler ve tarayıcının standart modunu seçmesine yardım eder. Örneğin bir kutunun genişliği eski uyumluluk davranışında farklı hesaplanabilir ve tasarım kayabilir.

Eksik olduğunda

Tarayıcı geriye uyumluluk için Quirks Mode'a geçebilir. Bazı CSS ölçüleri ve yerleşimler beklenenden farklı davranır. Quirks Mode eski siteleri çalıştırmak için vardır; modern bir projede beklediğimiz kuralların uygulanmasını istemeyiz.

Gerçek etki

Sayfanın mutlaka çökmesi veya işlemcinin belirgin yorulması gerekmez; temel risk tutarsız görüntüdür. Bu satırı ilk sıraya koymak kolay bir önlemdir: `<!DOCTYPE html>`.

Semantik HTML niin kullanılır?

Bölümlerin anlamı

<header>, <nav>, <main> ve <article> gibi öğeler içeriğın görevini belirtir; yalnızca görsel düzen oluşturmaz. Örneğın <nav> içinde menü bağlantılarını tutmak, ekran okuyucunun kullanıcıya gezinme bölümünü duyurmasını sağlar.

Erişilebilirlik

Tarayıcı bu anlamı erişilebilirlik ağacına yansıtır. Ekran okuyucular kullanıcıya bölümler arasında gezinme imkânı sağlayabilir. Bu bilgi HTML öğelerinden tarayıcı aracılığıyla erişilebilirlik arayüzlerine iletilir.

Arama motorları

Semantik yapı içeriğın anlaşılmasına yardımcı olur; tek başına üst sıralarda görünmeyi garanti etmez. Başlıkların mantıklı sırada olması ve bağlantı metinlerinin açık yazılması da semantik etiketler kadar önemlidir.

CSS sayfaya nasıl uygulanır?

Ayrı sorumluluklar

HTML içeriğin yapısını, CSS görünümünü tanımlar. Böylece aynı içerik farklı ekran boyutlarına göre biçimlenebilir. Örneğin birden fazla sayfada aynı style.css dosyasını kullanarak renkleri tek yerden değiştirebilirsin.

CSSOM ve çizim

Tarayıcı stil kurallarını işler, öğeler için hesaplanan stilleri bulur, yerleşimi hesaplar ve pikselleri çizer. DOM ile stil kuralları birlikte değerlendirilir; ardından tarayıcı her kutunun boyutunu ve yerini hesaplar.

Render blocking

Normal CSS dosyaları ilk çizimi bekletebilir. Bu bekleme stilsiz içeriğin anlık görünmesini azaltır; her sayfada mutlaka beyaz ekran oluşmaz. İlk çizimin beklemesi, tarayıcının kullanıcıya yanlış yerleşimli bir ara görüntü göstermesini azaltabilir.

Box Model ve yeniden hesaplama

Kutu modeli

İçerik alanı, padding, border ve margin öğenin ekrandaki alanını ve komşularına uzaklığını belirler. Örneğin padding düğmenin içindeki yazı ile kenarlığı açar; margin düğmeyi yanındaki öğelerden uzaklaştırır.

Layout / reflow

Genişlik gibi bir özellik değişirse tarayıcı bazı öğelerin konumunu yeniden hesaplayabilir. Etki değişikliğin kapsamına bağlıdır. Bir kartın genişliğini artırmak yanındaki kartların yerini de değiştirebilir; tarayıcı etkilenen alanı tekrar hesaplar.

Compositing ve GPU

transform ve opacity gibi animasyonlar uygun koşullarda compositing katmanında işlenebilir. GPU kullanımı otomatik ve koşulsuz değildir. Akıcı animasyon için transform tercih edilebilir; yine de gerçek performansı tarayıcı araçlarıyla ölçmek gerekir.

04 / JavaScript motoru

Etkileşim, olaylar ve asenkron işlemler

JavaScript kodu nasıl çalışır?

Motor

Tarayıcıdaki JavaScript motoru kaynak kodu ayrıştırır, çalıştırılabilir temsillere dönüştürür ve kodu yürütür. Örneğin bir düğmeye basılınca çalışan fonksiyon da aynı motor tarafından çözülüp yürütülür.

JIT

Modern motorlar sık çalışan kodu çalışma sırasında optimize edip makine koduna derleyebilir. Her satırın aynı biçimde JIT derlendiğini varsaymayız. Motor, sık tekrar eden bir kodu optimize ederek sonraki çalıştırmalarını hızlandırabilir; her kod aynı kazancı sağlamaz.

Kullanım

Değişkenler veriyi tutar; koşullar karar verir; fonksiyonlar tekrarlanan işleri bir araya getirir. Böylece kullanıcı eylemine göre metin değiştirmek veya veriyi yeniden göstermek mümkün olur.

Call Stack ve arayüzün donması

Çağrı yığını

Fonksiyon çağrıları stack'e eklenir; biten çağrılar çıkar. Uzun süren senkron bir görev ana iş parçacığını meşgul edebilir. Örneğin milyonlarca kez çalışan bir döngü bitene kadar aynı iş parçacığı yeni tıklamaları işleyemeyebilir.

Ekrandaki etki

Ana iş parçacığı yoğunken kullanıcı tıklamasına yanıt ve ekran güncellemesi gecikebilir. Bu sırada kullanıcı düğmeye basar ama sonuç gecikir; sorun düğmenin bozuk olmasından değil yoğun görevden kaynaklanır.

Nüans

JavaScript ana sayfa bağlamında çoğunlukla tek iş parçacığında çalışır; Web Worker ayrı iş parçacığında kod çalıştırabilir. Büyük hesapları parçalara bölmek veya uygun işi Web Worker'a taşımak arayüzün yanıtını koruyabilir.

Event Loop: Beklerken sayfa neden çalışır?

Asenkron işlem

fetch veya zamanlayıcı gibi işler tarayıcının ilgili mekanizmalarına devredilir; JavaScript sırf yanıt beklemek için stack'i bloke etmez. Örneğin fetch isteği sürerken kullanıcı sayfayı kaydırabilir; sonuç geldiğinde ilgili kod daha sonra çalışır.

Kuyruk ve olay döngüsü

İşlem hazır olduğunda görevi uygun kuyruğa alınır. Event loop, stack uygun olduğunda sıradaki işi çalıştırır. Event loop, bekleyen işleri belirli kurallarla sıraya alır; tüm işlemler aynı anda ana iş parçacığında çalışmaz.

Sınır

Ağ beklemek sayfayı dondurmaz; uzun bir senkron hesaplama yine kullanıcı arayüzünü geciktirebilir. Bu mekanizma hesaplama yükünü sihirli biçimde ortadan kaldırmaz; ağır bir döngü yine ana iş parçacığını kaplar.

JavaScript ve DOM ilişkisi

Tarayıcı API'si

DOM, JavaScript dilinin kendi parçası değildir; tarayıcı document gibi API'lerle ona erişim sağlar. Örneğin `document.querySelector('#sonuc')` tarayıcının sunduğu DOM API'siyle ilgili öğeyi bulur.

Değişiklik

Bir metni veya sınıfı değiştirince tarayıcı gereken stil, yerleşim ve çizim işlerini yeniden yapabilir. Metin değişikliği bazen yalnızca boyamayı, boyut değişikliği ise yerleşimi de yeniden hesaplamayı gerektirir.

Performans

Her DOM erişimi başlı başına pahalı değildir. Çok sık ve ölçümü zorlayan değişiklikler performans sorunu yaratabilir. Gereksiz tekrarlı ölçüm ve değişiklikleri azaltmak, özellikle büyük sayfalarda daha akıcı davranış sağlayabilir.

Küçük bir etkileşim örneği

HTML

`<button id="btn">Tıkla</button>` ve `<p id="sonuc">Hazır</p>` öğelerini ekle. İlk satırdaki id değerleri JavaScript'in hangi düğme ve paragrafla çalışacağını gösteren benzersiz işaretlerdir.

JavaScript

```
document.querySelector('#btn').addEventListener('click', () =>
{ document.querySelector('#sonuc').textContent = 'Merhaba!'; });
```

`addEventListener` bir tıklama gerçekleştiğinde çağrılacak fonksiyonu kaydeder; kod düğmeye basılmadan metni değiştirmez.

Ne gerçekleşir?

Tıklama olayı fonksiyonu tetikler. Fonksiyon DOM'daki paragraf metnini değiştirir. `textContent` yeni metni paragrafın içine yerleştirir ve değişiklik sayfayı yeniden yüklemeyi gerektirmez.

05 / Yayınlama

Yerel projeyi herkesin erişebileceği bir siteye dönüştürme

Alan adı ve DNS yayılımı

Alan adı

Kullanıcının yazdığı okunabilir ad DNS kayıtlarıyla sitenin hizmet verdiği adrese bağlanır. Örneğin yeni sunucuya geçince alan adının DNS kaydını değiştirirsin; kullanıcı yine aynı adı yazar.

Değişiklik ve önbellek

DNS kaydı değişince önbelleklerde eski yanıt TTL süresi dolana kadar bulunabilir; farklı kullanıcılar bir süre farklı sonuç görebilir. Bazı çözümleyiciler TTL bitene kadar eski IP'yi tuttuğundan geçiş herkes için aynı saniyede görünmez.

Yayılım ifadesi

'Küresel yayılım' çoğunlukla önbelleklerin güncellenmesini anlatır; her DNS sunucusuna aynı anda veri kopyalanması değildir. Geçişten önce TTL planlamak ve eski sunucuyu kısa süre açık tutmak kesintiyi azaltabilir.

CDN: İçerik kullanıcıya yaklaşır

Uç noktalar

CDN statik dosyaların kopyasını farklı bölgelerdeki edge sunucularda tutabilir. Örneğin İstanbul'daki kullanıcı görseli kıtalar arası kaynak yerine yakındaki edge noktasından alabilir.

Gecikme

Uygun durumda kullanıcı yakındaki önbellekten yanıt alır ve ağ mesafesi azalır. Dosya önbellekteyse CDN yanıtı hızlandırır; değilse önce kaynak sunucudan alıp sonrakiler için saklayabilir.

Sınır

Her dosya her edge sunucuda her zaman hazır olmayabilir. İlk istek kaynak sunucuya gidebilir. Dinamik veya kişiye özel içerik için önbellekleme kuralları ayrıca düşünülmelidir.

CI/CD ile dağıtım

CI

Depoya kod gönderildiğinde otomatik kontroller, testler ve derleme çalışabilir. Örneğin GitHub'a gönderilen kod için otomatik bir iş akışı önce testleri çalıştırıp sonra yayın dosyalarını hazırlayabilir.

CD

Başarılı bir sürüm barındırma ortamına otomatik veya onayla dağıtılabilir. Test başarısızsa dağıtımı durdurmak, bozuk bir sürümün canlı siteye çıkmasını önlemeye yardımcı olur.

Önemli ayrım

GitHub Pages, Vercel ve Netlify farklı dağıtım akışları sunar. Otomatik test ve kesintisiz geçiş projenin kurulumuna bağlıdır. Hangi dala gönderimin yayın başlatacağı ve onay gerekip gerekmediği projeye göre belirlenir.

HTTPS ve TLS güvenliği

Sertifika

Tarayıcı, sunucunun sunduğu sertifika zincirini ve alan adı eşleşmesini doğrular. Örneğin tarayıcı adres çubuğundaki alan adıyla sertifikadaki ad uyuşmazsa kullanıcıyı uyarır.

Şifreli oturum

TLS anahtar değişimiyle bağlantıyı şifreler ve verinin değiştirilmesini tespit etmeye yardımcı olur. Ağdaki üçüncü bir kişi trafiği görebilse bile doğru kurulan TLS içeriği okunamaz ve değişiklikler saptanır.

Sınırlar

Doğru yapılandırılmış HTTPS ağdaki dinleyicilere karşı güçlü koruma sağlar; sahte siteye gitmek veya cihazın ele geçirilmesi gibi riskleri tek başına çözmez. Fakat kullanıcı saldırganın benzer isimli sitesine kendi isteğiyle girerse geçerli HTTPS bağlantısı tek başına korumaz.

Baştan sona: İlk web sitesi

Hazırla

Editörde proje klasörünü oluştur; yerel sunucuda aç ve tarayıcı araçlarıyla incele. Örneğin index.html, style.css ve scripts/main.js dosyalarını tek proje içinde tutup localhost'ta birlikte açarsın.

Geliştir

HTML yapısını kur, CSS ile düzenle, JavaScript ile bir tıklama etkileşimi ekle. Bir düğmenin metin değiştirmesi HTML ögesi, CSS görünümü ve JavaScript olayının birlikte çalıştığını gösterir.

Yayınla

Git geçmişine kaydet, barındırma hizmetine dağıt ve HTTPS üzerinden çalışan adresi test et. Canlı adresi telefonda da açıp görselleri, bağlantıları ve tarayıcı konsolundaki hataları kontrol edersin.